

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF
TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Internet of Things Application Development

Assignment Report

Smart Environmental Monitoring System

Dr. Le Trong Nhan

HO CHI MINH CITY, DECEMBER 2025



Member List & Workload

No.	Full Name	Student ID	Details
1	Trần Minh Hiếu	2252106	Task 5 ML, Task 6
2	Nguyễn Đức Tài	2252723	Board Task 1, 2, 3, 4
3	Nguyễn Thủy Tiên	2252806	Task 5 Hand Gesture, Task 6

Contents

Introduction and Objectives	6
1 Core IOT Platform and Dashboard Design	10
1.1 Create Core IOT Account	10
1.1.1 Registration Process	10
1.1.2 Device Provisioning	13
1.2 Telemetry Data Transmission	15
1.3 Dashboard Implementation	17
1.3.1 Dashboard Layout Architecture	18
1.3.2 Status Indicators and Device Metrics	18
1.3.3 Remote Control Panel	18
1.3.4 Environmental Data Visualization	19
1.3.5 Alarm Management System	20
1.3.6 Geographical Fleet Tracking	20
1.3.7 Widget Configuration Best Practices	21
1.4 Summary	22
2 Task 1 - Single LED Blink with Temperature Conditions	24
2.1 LED Behavior Summary	25
2.2 Semaphore-Based State Handling	26
2.3 LED Task Loop	27
2.4 Visualizing LED Response	28
3 Task 2 - NeoPixel LED Control Based on Humidity	29
3.1 Humidity-to-Color Map	30
3.2 State-Based Effects	30
3.3 NeoPixel Task Loop	31
3.4 Visual Feedback Summary	33
4 Task 3 - Temperature and Humidity Monitoring with LCD Display	34
4.1 OLED Layout Modes	35
4.2 State Selection Flow	35
4.3 Queue and Semaphore Data Flow	36



5	Web Server Interface and Cloud Integration	40
5.1	Embedded Web Server Design	40
5.1.1	Access Point Configuration	40
5.1.2	User Interface Implementation	41
5.1.3	Web Communication Protocols	41
5.1.4	WebSocket Communication	42
5.2	Cloud Integration and Data Publishing	42
5.2.1	Station Mode Connectivity	43
5.2.2	Data Publishing to CoreIOT	43
5.2.3	Integration Improvements	44
5.3	Data Logging and CSV Export	44
5.3.1	CSV File Structure	44
5.3.2	System State Classification	45
6	Chapter 6	46
6.1	Section 1	46
6.2	Section 2	46
6.3	Summary	46
7	Task 6 - Data Publishing to CoreIOT Cloud Server	47
7.1	Create Core IOT Account	47
7.1.1	Registration Process	47
7.2	Accessing CoreIOT via Web Server	50
7.2.1	Device Provisioning	51
7.3	Telemetry Data Transmission	53
7.4	Dashboard Implementation	55
7.4.1	Dashboard Layout Architecture	56
7.4.2	Status Indicators and Device Metrics	56
7.4.3	Remote Control Panel	56
7.4.4	Environmental Data Visualization	57
7.4.5	Alarm Management System	58
7.4.6	Widget Configuration	60
7.5	Summary	60
8	Chapter 8	61
8.1	Section 1	61
8.2	Section 2	61
8.3	Summary	61



9 Chapter 9	62
9.1 Section 1	62
9.2 Section 2	62
9.3 Summary	62
Conclusion	63

Introduction and Objectives

Introduction

The Internet of Things (IoT) represents a transformative paradigm in modern computing, enabling physical devices to collect, exchange, and act upon data through network connectivity. As embedded systems become increasingly accessible and cloud platforms offer robust infrastructure for device management, understanding IoT architecture has become essential for engineering students. This project serves as a comprehensive learning exercise to explore the fundamental concepts and practical implementation of an IoT system.

Our project focuses on building a multi-sensor monitoring and control system using the YOLO Uno development board. The system integrates various sensors to collect environmental data and transmits this information to a cloud-based IoT platform for visualization and analysis. More importantly, the system implements bidirectional communication, allowing users to remotely control actuators connected to the device through an internet-accessible dashboard.

The core principle of IoT lies in the seamless connection between the physical and digital worlds. By developing this project, we gain hands-on experience with the complete IoT technology stack, from low-level sensor interfacing and embedded firmware development to cloud platform configuration and real-time data visualization. This holistic approach provides valuable insights into how modern connected systems operate and interact.

Objectives

The primary objective of this project is to design and implement a comprehensive environmental monitoring and control system that demonstrates the core capabilities of IoT technology. This system serves as an educational platform to understand how multiple sensors can be integrated with cloud platforms to enable real-time monitoring and remote control of physical devices. The environmental control system we build incorporates various sensors to continuously monitor environmental conditions and provides the ability to remotely control actuators such as fans, LEDs, and water pumps through an internet-connected dashboard. This project aims to showcase the complete IoT development cycle, from hardware integration and embedded firmware programming to cloud platform configuration and data visualization.

The primary objectives of this project are structured around the fundamental aspects of IoT system development:



Remote Device Control via Internet

The central objective of this project is to demonstrate remote device control through internet connectivity. We implement a system where actuators such as fans, LEDs, and water pumps can be controlled from anywhere with an internet connection. This showcases the core value proposition of IoT: the ability to monitor and control physical devices regardless of geographical location.

Sensor Integration and Data Acquisition

We integrate multiple sensors to create a comprehensive data acquisition system. The project incorporates:

- Temperature and humidity sensor (DHT11) for environmental monitoring
- Light sensor for ambient light level detection
- Soil moisture sensor for humidity measurement
- Flame sensor for fire detection scenarios

This multi-sensor approach provides experience in interfacing with different sensor types and managing concurrent data streams within a real-time operating system environment.

Cloud Platform Integration

We utilize the Core IOT platform (based on ThingsBoard) to establish device-to-cloud communication. The objectives include:

- Configuring MQTT-based telemetry transmission
- Implementing Remote Procedure Calls (RPC) for bidirectional communication
- Designing interactive dashboards for data visualization
- Understanding authentication and security mechanisms in IoT deployments

Over-the-Air (OTA) Firmware Update

A key objective is implementing OTA firmware update capability, enabling remote firmware deployment without physical access to the device. This feature is essential for IoT devices deployed in the field, allowing:

- Remote bug fixes and feature updates

- Reduced maintenance costs and downtime
- Seamless deployment of new functionality

We implement a web-based OTA interface using the ElegantOTA library, allowing firmware uploads through the device's local network interface.

Real-Time Operating System Implementation

The firmware is developed using FreeRTOS to manage multiple concurrent tasks efficiently. This provides practical experience with:

- Task scheduling and priority management
- Inter-task communication using queues and semaphores
- Thread-safe data access patterns
- Resource sharing in embedded systems

Report Structure

This report is organized into four main chapters, each covering a distinct aspect of the IoT system development:

1. Chapter 1: Core IOT Platform and Dashboard Design

This chapter covers the setup of the cloud platform, including account creation, device provisioning, telemetry transmission, and dashboard configuration with various widget types. It details the implementation of MQTT communication, RPC callbacks for remote control, and the design of interactive visualization dashboards.

2. Chapter 2: Hardware Architecture and Firmware Development

This chapter details the hardware components, circuit design, and the FreeRTOS-based firmware implementation for sensor data acquisition and actuator control. It includes the integration of multiple sensors, the real-time operating system task structure, and inter-task communication mechanisms.

3. Chapter 3: TinyML Integration

This chapter explores the integration of machine learning capabilities on the edge device for intelligent decision-making based on sensor data patterns. It covers model training, quantization, deployment, and inference on the YOLO Uno platform.



4. Chapter 4: System Integration and Testing

This chapter presents the complete system integration, testing procedures, and evaluation of the implemented IoT solution. It includes performance analysis, reliability testing, and discussion of the system's capabilities and limitations.

Chapter 1

Hardware and Platform Overview

This chapter provides an overview of the hardware and cloud platform used in our project. We introduce the Yolo Uno development board and the Core IOT platform, outlining the setup process and dashboard design. Detailed firmware implementation and code explanations are reserved for Chapter 7.

1.1 Introduction to Yolo Uno

The Yolo Uno is a versatile development board designed for IoT applications, featuring the powerful ESP32 microcontroller. It integrates essential components such as Wi-Fi, Bluetooth, and various onboard sensors/actuators, making it an ideal choice for rapid prototyping.

Key features of the Yolo Uno include:

- **ESP32-WROOM-32E:** Dual-core processor with Wi-Fi and Bluetooth connectivity.
- **Onboard Peripherals:** Includes NeoPixel LEDs, buttons, and Grove connectors for easy sensor expansion.
- **USB-C Interface:** For programming and power.
- **Compact Form Factor:** Suitable for embedded projects.



Figure 1.1: The Yolo Uno development board.

1.2 Core IOT Platform Setup

Core IOT is a cloud-based IoT platform built on the ThingsBoard framework. We selected this platform for our project due to its open-source nature and comprehensive feature set for device management and data visualization.

1.2.1 Registration Process

We accessed the Core IOT platform at <https://coreiot.io> and clicked the "Start Now" button to begin registration (Figure ??).



assets/Coreiot.png

Figure 1.2: Core IOT homepage with the "Start Now" button for registration.

The platform offers multiple login options through OAuth providers. We chose to authenticate using a Google account for convenience (Figure ??).



assets/Login.png

Figure 1.3: Login interface with OAuth provider options.

After successful authentication, we were assigned the Tenant Administrator role, which provides full access to device management and dashboard creation features (Figure ??).



assets/platform.png

Figure 1.4: Dashboard view after login showing Tenant Administrator role.

1.2.2 Device Provisioning

With the account ready, we proceeded to register our ESP32-based device on the platform. In the "Devices" section, we created a new device and obtained an access token. This token is required for the device to authenticate when sending telemetry data via MQTT.

To verify the connection and data transmission capabilities, we utilized a Python script to simulate the IoT device behavior before deploying it to the actual hardware. It is critical to note that we specifically selected the `paho-mqtt` library version 1.6.1 for this implementation,

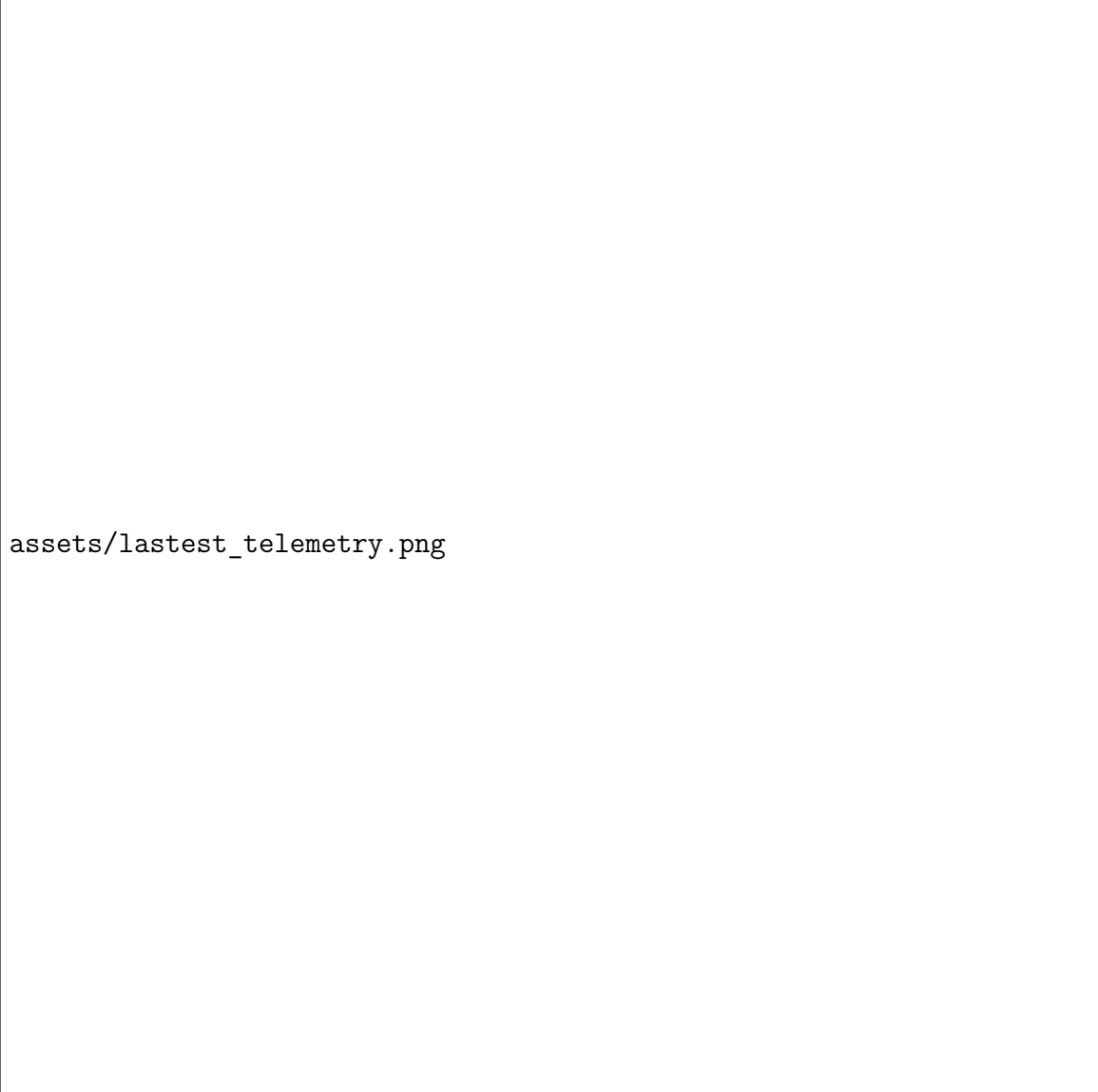


as newer versions (e.g., 2.0) introduce breaking changes in the authentication handling that are incompatible with our current configuration.

1.3 Telemetry Data Transmission

After configuring the device credentials, we developed a firmware routine to format sensor data into a JSON structure compliant with the Core IOT platform's requirements. The main task runs in a FreeRTOS task, waiting for WiFi connectivity before attempting to connect to Core IOT. It then enters a loop that publishes telemetry data every 10 seconds.

We verified the data ingestion by navigating to the "Latest Telemetry" tab within the device details page on the Core IOT portal. This interface allows us to confirm that the JSON payload is being parsed correctly and that values for temperature and humidity are updating dynamically as sent by the device.



assets/lastest_telemetry.png

Figure 1.5: The "Latest Telemetry" tab displaying real-time data received from the device.

1.4 Dashboard Implementation

One of the primary advantages of the Core IOT platform is its capability to visualize data through customizable dashboards. We created a comprehensive monitoring dashboard named *Control and Monitoring center* to serve as the central interface for real-time system oversight and actuator control. The dashboard employs a multi-panel layout that organizes information hierarchically, prioritizing critical control functions and environmental data visualization.

1.4.1 Dashboard Layout Architecture

We designed the dashboard with a responsive grid layout divided into three primary horizontal sections. The top section contains system status indicators and actuator controls, providing immediate access to the most frequently used functions. The middle section displays real-time sensor data through time-series charts and gauge widgets, enabling trend analysis and anomaly detection. The bottom section presents an alarm management table and geographical visualization, supporting incident tracking and multi-device deployment scenarios.

This hierarchical organization follows established human-computer interaction principles, placing high-priority interactive elements within the primary visual field while maintaining comprehensive data visibility through strategic use of white space and visual grouping.

1.4.2 Status Indicators and Device Metrics

The dashboard header displays two key performance indicators using card widgets. The first card shows the active device count, labeled **Devices**, which indicates the number of ESP32 devices currently connected to the platform. In our deployment, this displays a value of 1, confirming that our primary environmental monitoring device maintains an active MQTT connection.

Adjacent to the device count, we positioned a **Total** card that aggregates a custom metric across all connected devices. This counter widget can be configured to track various cumulative statistics such as total alarm count, cumulative uptime, or aggregated sensor readings. The card widgets utilize ThingsBoard's entity aggregation functions, which query the time-series database to compute values based on configurable time windows.

To the right of the status cards, we integrated a WiFi quality indicator using an icon widget. This widget dynamically changes color based on the RSSI (Received Signal Strength Indicator) value transmitted in the device telemetry. Strong signal strength displays a green icon, while degraded connectivity transitions to yellow or red, providing immediate visual feedback on network health without requiring detailed inspection of raw telemetry values.

1.4.3 Remote Control Panel

We implemented a dedicated control panel section containing three RPC toggle switches for actuator management: Water Pump, Fan, and NeoLed (RGB LED strip). Each switch widget is configured with bidirectional RPC methods to both read the current state and issue control commands.

The switch configuration employs getter and setter RPC methods. When the dashboard loads, each switch automatically invokes its corresponding getter method (`getFanStatus`, `getWaterPumpStatus`, `getLedStatus`) to synchronize the UI state with the actual device state.

This prevents desynchronization scenarios where the dashboard might display **OFF** while the device is actively driving the actuator.

When a user toggles a switch, the dashboard transmits an RPC request containing the method name and the desired boolean state. For example, clicking the Fan switch from OFF to ON triggers an RPC call to `setFanStatus` with parameter `true`. The device firmware processes this command, actuates the corresponding GPIO pin, and returns an `RPC_Response` to acknowledge successful execution.

We observed that timeout errors occasionally occur when the device is processing intensive tasks or when network latency exceeds the default RPC timeout threshold (5 seconds). In such cases, the switch widget displays a *Request timeout* modal, prompting the user to retry the command. To mitigate this issue, we optimized the RPC callback functions to execute GPIO operations quickly and defer any blocking operations to separate FreeRTOS tasks.

1.4.4 Environmental Data Visualization

The central focus of the dashboard is the environmental monitoring section, which utilizes time-series charts to visualize temperature and humidity trends. We configured a line chart widget with dual Y-axes, where temperature is plotted in degrees Celsius on the left axis and relative humidity percentage on the right axis. This dual-axis configuration enables direct visual correlation between the two variables, which is essential for identifying environmental patterns such as evaporative cooling effects.

The chart widget subscribes to the `temperature` and `humidity` telemetry keys and maintains a rolling time window of the last hour by default. Users can adjust this window through the chart's time range selector, enabling both real-time monitoring and historical analysis. We customized the chart appearance with distinct colors: red for temperature and blue for humidity, following conventional meteorological visualization standards.

Below the time-series chart, we positioned a second chart dedicated to soil moisture monitoring. This widget displays the `moisture` telemetry key, providing agricultural and plant care applications with a clear view of substrate hydration levels over time. The moisture chart uses a green color scheme and includes threshold lines at 30% (dry) and 70% (saturated) to guide irrigation decisions.

Adjacent to the charts, we implemented a large numeric display widget showing the current light level in lux. This widget is configured with the `light` telemetry key and formatted to display values with scientific notation (e.g., 65000 m², though the unit should technically be lux). The large font size makes this value readable at a glance, which is particularly useful for automated greenhouse or indoor farming applications where lighting optimization is critical.

1.4.5 Alarm Management System

We integrated a comprehensive alarm table widget in the lower section of the dashboard to track environmental anomalies and system events. The alarm system operates through rule chains configured in the ThingsBoard Rule Engine, which continuously evaluate incoming telemetry against predefined threshold conditions.

The alarm table displays several critical columns:

- **Created time:** Timestamp when the alarm was triggered, enabling chronological incident analysis
- **Originator:** The entity that generated the alarm (typically *Assignment* for our configured rule chain)
- **Type:** Classification of the alarm event (e.g., *Environment Warning*)
- **Severity:** Criticality level (Warning, Critical, or Fire Alert)
- **Status:** Current state of the alarm (Active, Acknowledged, Cleared, Unacknowledged)
- **Assignee:** User or group responsible for incident response

In our current deployment, the alarm table shows multiple *Environment Warning* entries triggered on December 3, 2025. These alarms are generated when temperature exceeds 33°C, drops below 15°C, or when humidity falls outside the 40–70% comfort range. The severity levels are color-coded: green for acknowledged alarms, yellow for warnings, and red for critical events.

We configured the alarm rules using ThingsBoard's Rule Chain editor, which provides a visual programming interface for defining conditional logic. For example, the temperature threshold rule consists of a filter node that evaluates `temperature > 33`, connected to an action node that creates an alarm entity with severity *Warning* and type *Environment Warning*. The rule chain also sends WebSocket notifications to the dashboard, triggering real-time alert pop-ups.



assets/rule_chain.png

Figure 1.6: Rule Chain configuration for alarm processing and notification.

The alarm table supports interactive management features. Operators can click individual alarms to view detailed information, acknowledge events to indicate they are being addressed, or assign alarms to specific team members for resolution. This functionality is particularly valuable in multi-user deployments where incident response requires coordination among maintenance personnel.

1.4.6 Geographical Fleet Tracking

At the bottom of the dashboard, we incorporated an Open Street Map widget to provide spatial context for device deployments. While our current system operates with a single device, the map widget demonstrates scalability for distributed sensor networks across multiple locations.

The map configuration requires telemetry data to include `latitude` and `longitude` fields. These coordinates can be hardcoded in the firmware for fixed installations or dynamically obtained from GPS modules for mobile deployments. Each device appears as a marker on the map, and clicking a marker displays a tooltip containing key telemetry values such as current temperature, humidity, and device status.

We customized the map widget's datasource settings to ensure proper data key mapping when multiple devices are present. By default, ThingsBoard appends numeric suffixes to duplicate data keys (e.g., `latitude_1`, `latitude_2`), but we configured explicit aliases to maintain consistent key names across all devices. This prevents widget configuration errors that would occur if the dashboard expected `latitude` but received `latitude_1`.

1.4.7 Widget Configuration Best Practices

Throughout the dashboard design process, we identified several critical configuration requirements:

Data Key Consistency: The telemetry keys defined in the firmware (`temperature`, `humidity`, `light`, etc.) must exactly match the data keys configured in each widget. Case sensitivity and spelling errors will result in widgets displaying *No data* even when the device is actively transmitting.

RPC Method Naming: Switch widgets require precise RPC method names that correspond to the registered callbacks in the firmware. We standardized on a naming convention using camelCase with *set* and *get* prefixes (e.g., `setFanStatus`, `getFanStatus`).

Widget Refresh Rates: We configured chart widgets to poll for new data every 5 seconds, balancing real-time responsiveness with server load. Faster refresh rates increase database query frequency, which can impact platform performance in deployments with hundreds of devices.

Color Coding and Visual Hierarchy: We applied consistent color schemes throughout the dashboard: green for normal conditions, yellow for warnings, and red for critical alerts. This visual language enables rapid situation assessment without requiring detailed reading of all numeric values.



assets/control_monitoring_dashboard.png

Figure 1.7: Complete Control and Monitoring center dashboard showing status cards, RPC switches, time-series charts, light level display, WiFi indicator, alarm table, and map widget.

1.5 Summary

In this chapter, we introduced the Yolo Uno hardware and the Core IOT cloud platform. We outlined the initial setup, including account registration, device provisioning, and the conceptual design of the monitoring dashboard. The specific software implementation, including MQTT communication and FreeRTOS tasks, will be detailed in Chapter 7.

Chapter 2

Task 1 - Single LED Blink with Temperature Conditions

The three following chapters describes the implementation of three real-time tasks that provide visual feedback based on sensor data using FreeRTOS. The system follows a producer-consumer architecture in which sensor tasks publish data to a queue, while actuator tasks control the LED, NeoPixel, and OLED display based on this data. Binary semaphores are used to signal system states such as Normal, Warning, Critical, and Fire Alert, while a mutex-protected queue ensures thread-safe data exchange. By using queues and semaphores instead of global variables, the design achieves safe, modular, and synchronized inter-task communication. Figure ?? shows how tasks communicate with each other.

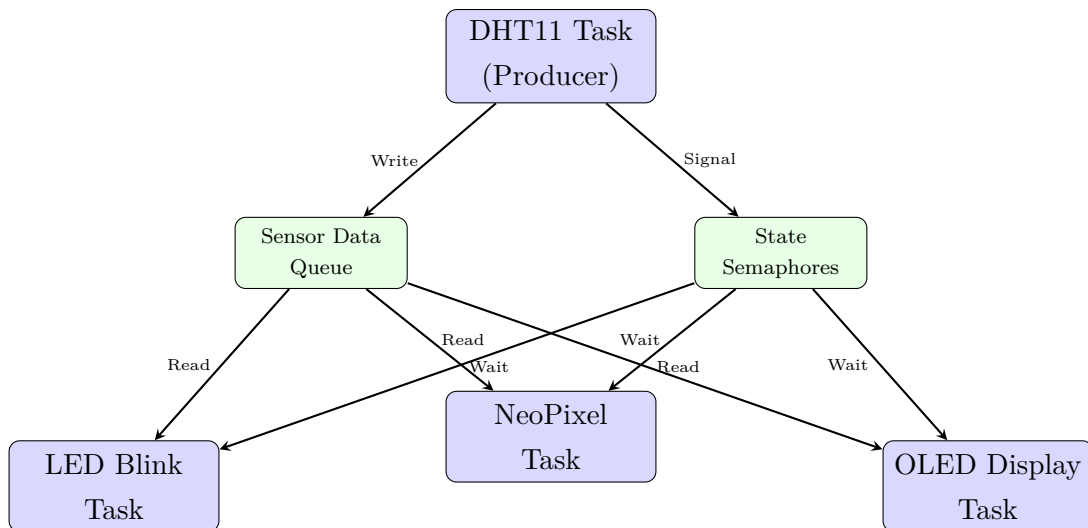


Figure 2.1: Task synchronization architecture showing producer-consumer pattern with queue and semaphore-based communication.

This chapter focuses on the implementation of a single LED blinking system controlled by temperature conditions to provide real-time visual feedback of environmental status. The objective of this task is to translate temperature and flame sensor readings into distinct LED blinking patterns that reflect different system states, including Normal, Warning, Critical, and Fire Alert. By utilizing separate normal and alert LEDs with clearly defined behaviors,



the system enables users to quickly identify hazardous conditions without requiring a display interface. This chapter details the design logic, state-based control strategy, and the practical realization of temperature-driven LED signaling.

2.1 LED Behavior Summary

We implemented a dual-LED indicator system to show environmental conditions visually. We connected two LEDs to GPIO pins 48 (normal LED) and 47 (alert LED). The blinking pattern changes based on the system state, which depends on temperature thresholds and flame detection.

The LED task responds to four distinct system states, each triggering a unique visual pattern. This multi-state design ensures that operators can assess system status at a glance without consulting the dashboard. Table ?? summarizes the complete behavior specification.

Table 2.1: LED blinking behaviors mapped to temperature conditions and system states.

Temperature Range	Condition	Normal LED (GPIO 48)	Alert LED (GPIO 47)	Blink Period
<15°C	Cold	Slow blink (ON/OFF)	OFF	1000 ms
15–33°C	Normal Range	Normal blink	OFF	500 ms
33–40°C	Hot (Warning)	Fast blink	OFF	200 ms
>40°C	Critical	Very fast blink	OFF	100 ms
Flame detected	Fire Alert	OFF	Solid ON (max brightness)	–



Figure 2.2: Dual-LED indicator system showing normal LED (GPIO 48) and alert LED (GPIO 47) mounted on the YOLO Uno board.

2.2 Semaphore-Based State Handling

The temperature monitoring task evaluates sensor readings and signals the appropriate state using binary semaphores. We defined four binary semaphores corresponding to the four system states: `xSemaphoreNormal`, `xSemaphoreWarning`, `xSemaphoreCritical`, and `xSemaphoreFireAlert`.

When a state transition occurs, the monitoring task first clears all state semaphores by taking them with a zero timeout, then gives only the semaphore corresponding to the new state. This ensures that exactly one state is active at any time, preventing ambiguous conditions. The LED task can then check which semaphore is available to determine the current state.

The state selection follows a strict priority order: **Fire Alert** > **Critical** > **Warning** > **Normal**. This ensures that if a fire is detected, the LED immediately switches to the solid ON alert mode, regardless of the current temperature.

Figure ?? illustrates the state machine governing LED behavior:

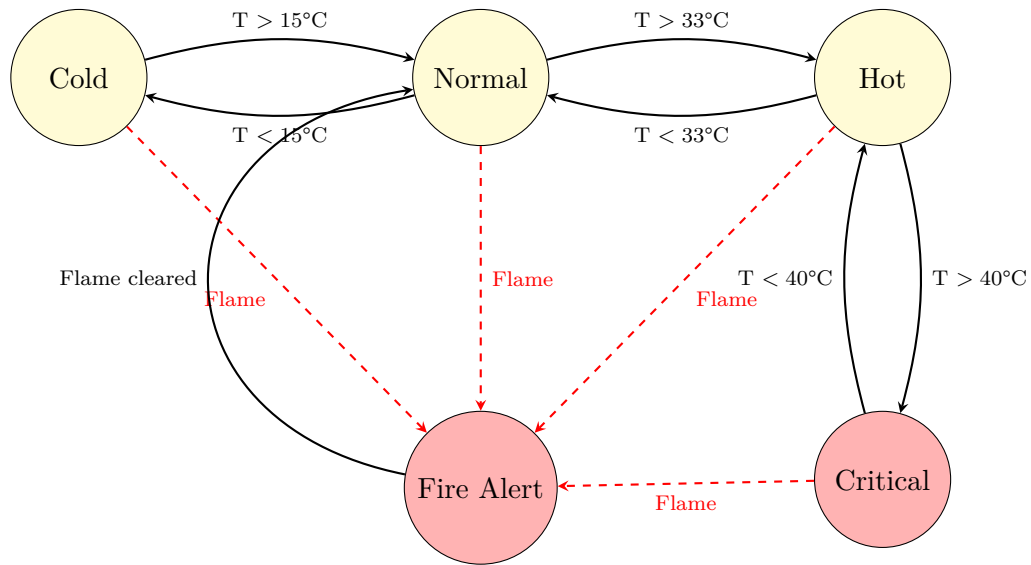


Figure 2.3: State machine for LED behavior showing transitions based on temperature thresholds and flame detection.

2.3 LED Task Loop

The LED task operates in an infinite loop, continuously monitoring the sensor queue and system state. The core logic follows this pseudocode structure:

```

TASK led_blinky():
    Initialize GPIO pins 48 and 47 as outputs
    Initialize local variables (blinkState, sensorData)

    WHILE TRUE:
        // Non-destructive queue read
        Peek sensor data from xSensorDataQueue
        Read current system state (thread-safe)

        IF state == FIRE_ALERT OR flame_detected:
            Set Alert LED to maximum brightness (GPIO 47 = HIGH)
            Set Normal LED off (GPIO 48 = LOW)
            Delay 100ms
        ELSE:
            Set Alert LED off (GPIO 47 = LOW)
            Toggle blinkState
            Set Normal LED according to blinkState
  
```

```

        Delay 500ms
    END WHILE
END TASK

```

The task uses `xQueuePeek()` to read sensor data without removing it from the queue, so multiple tasks can read the same data. The `getSystemState()` function safely reads the current state using a mutex. We set the blink period to 500 ms, which is fast enough to catch attention but not too fast to be annoying.

The semaphore-based state checking follows this logic: the task reads sensor data from the queue using a non-destructive peek operation, then calls `getSystemState()` which internally checks which state semaphore is available. If the state is `STATE_FIRE_ALERT`, the alert LED is set to maximum brightness and the normal LED is turned off. Otherwise, the normal LED blinks at 500 ms intervals while the alert LED remains off.

2.4 Visualizing LED Response

To better understand the system's dynamic response, we plotted the relationship between the measured temperature and the LED blinking frequency. The system exhibits a step-wise increase in urgency as temperature rises.

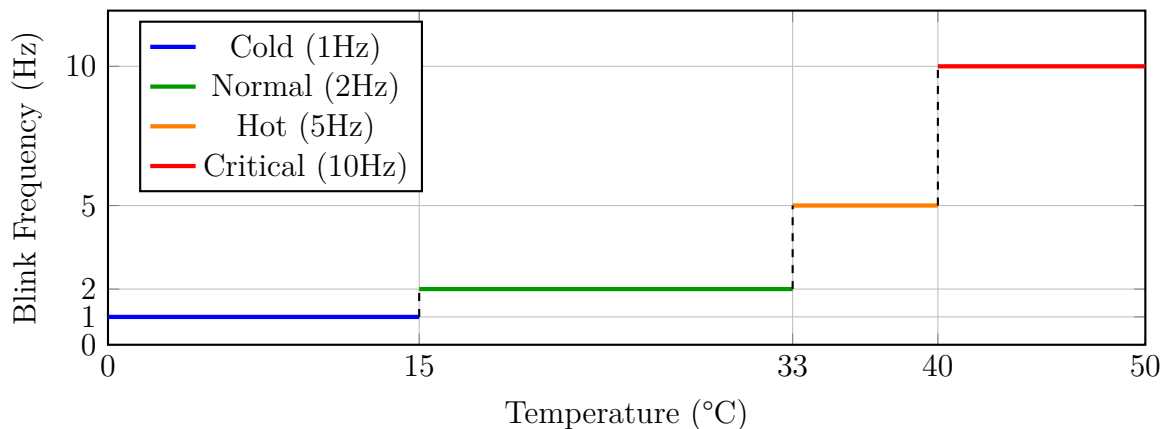


Figure 2.4: Temperature thresholds and LED blinking frequency.

Chapter 3

Task 2 - NeoPixel LED Control Based on Humidity

This chapter presents the design and implementation of a NeoPixel LED control system based on real-time humidity monitoring. The objective of this task is to provide intuitive visual feedback of environmental moisture conditions using color-coded RGB lighting. By mapping predefined humidity ranges to distinct LED colors, users can quickly interpret indoor environmental quality without relying on numerical readings. The system categorizes humidity conditions into different operational states such as Normal, Warning, and Critical, each represented by a specific color output. Through this approach, the NeoPixel LED serves as both a monitoring indicator and an early warning mechanism, enhancing overall system usability and environmental awareness.

3.1 Humidity-to-Color Map

We implemented a NeoPixel controller that changes color based on humidity levels. We chose colors that are easy to understand: green for good conditions, red for problems, and other colors for intermediate states.

Table 3.1: NeoPixel color mapping for different humidity ranges with RGB values.

Humidity Range	Condition	State	Color	RGB Values
<30%	Too Dry	Critical	Red	(255, 0, 0)
30–40%	Dry	Warning	Orange	(255, 100, 0)
40–70%	Ideal	Normal	Green	(0, 255, 0)
70–80%	Acceptable	Normal	Cyan	(0, 200, 255)
>80%	Too Humid	Warning	Purple	(150, 0, 255)

We created five color zones to give more detailed feedback. The 40–70% range shows green (ideal conditions), while other ranges show cyan (acceptable up to 80%), orange (dry warning), or purple (too humid above 80%). If humidity drops below 30%, we show red to indicate a critical condition.

3.2 State-Based Effects

3.3 Semaphore Mechanism and Priority

To ensure thread-safe state management, we utilize FreeRTOS binary semaphores as signaling flags. Each system state (Normal, Warning, Critical, Fire Alert) is represented by a unique binary semaphore.

3.3.1 Signaling Mechanism

The state transition logic follows a "Take-All, Give-One" approach. When the system state changes, the monitoring task first "takes" (clears) all four semaphores to ensure no conflicting signals exist. Immediately after, it "gives" (sets) only the semaphore corresponding to the new active state. This guarantees that at any given moment, exactly one state semaphore is available for consumer tasks to read.

3.3.2 Priority Hierarchy

The determination of which semaphore to set is governed by a strict priority hierarchy. This ensures that critical safety alerts always override minor warnings. The priority levels are defined as follows:

1. **Priority 1 (Highest) - Fire Alert:** Triggered immediately if the flame sensor detects fire. This state overrides all temperature and humidity readings.
2. **Priority 2 - Critical:** Triggered if temperature exceeds 40°C or humidity drops below 30%.
3. **Priority 3 - Warning:** Triggered if conditions are outside the comfortable range (Temp <15°C or >33°C, Humidity <40% or >80%).
4. **Priority 4 (Lowest) - Normal:** Active only when no other conditions are met (Temp 15–33°C, Humidity 40–80%).

The NeoPixel task respects this hierarchy by checking the system state before applying any color logic. If the state is Fire Alert, it ignores the humidity sensor entirely and displays the emergency red signal.

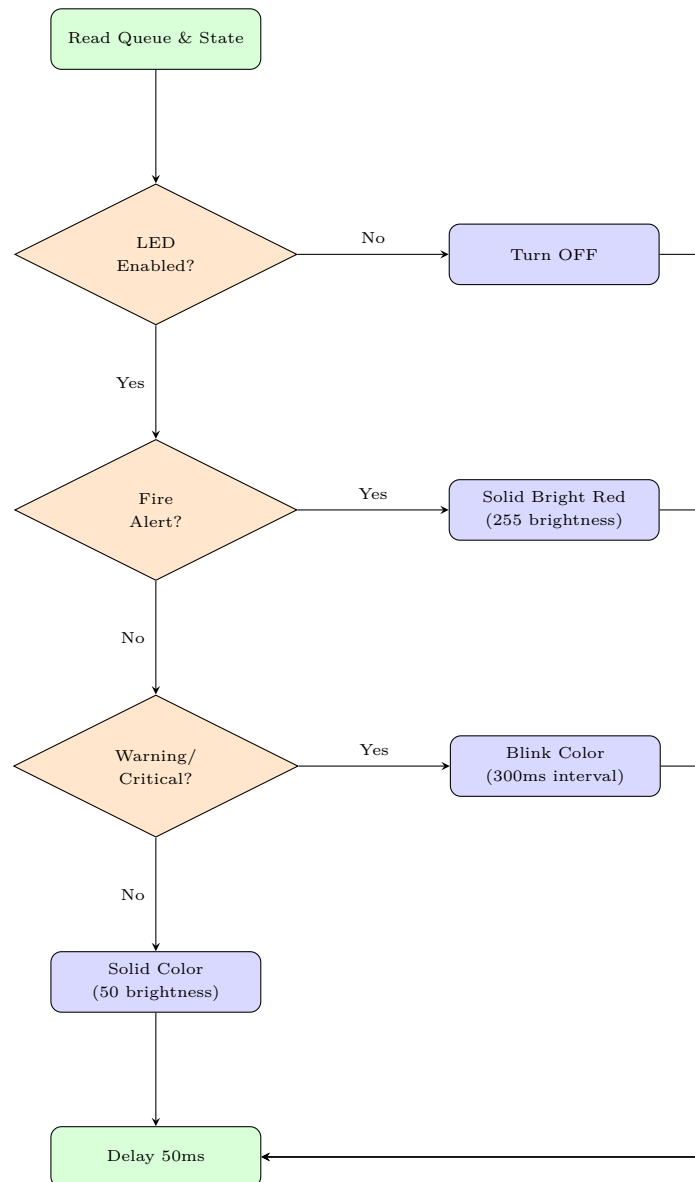


Figure 3.1: NeoPixel control flow showing decision logic based on LED enable status and system state.

3.4 NeoPixel Task Loop

The NeoPixel task implements a hierarchical priority system where fire alerts override all other conditions. The pseudocode structure demonstrates this layered decision logic:

```

FUNCTION getColorByHumidity(humidity):
    IF humidity < 30: RETURN Red(255,0,0)
    ELSE IF humidity < 40: RETURN Orange(255,100,0)
    ELSE IF humidity <= 60: RETURN Green(0,255,0)
    ELSE IF humidity <= 70: RETURN Cyan(0,200,255)
  
```

```
    ELSE: RETURN Purple(150,0,255)
END FUNCTION

TASK neo_blinky():
    Initialize NeoPixel strip on GPIO 45
    Set default brightness to 50

    WHILE TRUE:
        Read sensor data from queue
        Read current system state

        IF LED disabled (via RPC control):
            Turn off LED
            CONTINUE
        END IF

        IF state == FIRE_ALERT OR flame detected:
            Set brightness to 255 (maximum)
            Display solid bright red
        ELSE IF state == WARNING OR CRITICAL:
            Set brightness based on severity (70 or 100)
            Get color from humidity mapping
            Blink color at 300ms intervals
        ELSE (Normal state):
            Set brightness to 50 (power saving)
            Get color from humidity mapping
            Display solid color
        END IF

        Delay 50ms
    END WHILE
END TASK
```

The color selection function checks humidity ranges to pick the right color. We adjust brightness to save power during normal operation (brightness 50) and make alerts more visible (brightness 70 for warnings, 100 for critical). This makes critical states stand out even when using the same color.

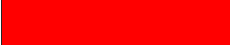
The semaphore-based state checking works as follows: the task first checks if the LED is

enabled via RPC control by reading from the sensor data queue. If disabled, the LED remains off. Otherwise, it calls `getSystemState()` to determine the current system state. Based on the state semaphore, the task selects the appropriate behavior: fire alert displays solid bright red at maximum brightness, warning/critical states blink the humidity-based color at 300 ms intervals, and normal state displays a solid color at reduced brightness.

3.5 NeoPixel Color Mapping

To provide a clear reference for the user, Table below summarizes the NeoPixel's visual behavior, including color codes and motion effects (Solid vs. Blinking) for each humidity range and system state.

Table 3.2: NeoPixel visual feedback summary showing color and motion for each condition.

Humidity	State	Color Name	Visual Color	Motion
<30%	Critical	Red		Blink
30–40%	Warning	Orange		Blink
40–70%	Normal	Green		Solid
70–80%	Normal	Cyan		Solid
>80%	Warning	Purple		Blink
Flame Detected	Fire Alert	Bright Red		Solid (Max Brightness)

Chapter 4

Task 3 - Temperature and Humidity Monitoring with LCD Display

This chapter presents the design and implementation of the temperature and humidity monitoring system integrated with an OLED display for real-time environmental visualization. The system continuously measures ambient conditions and dynamically updates the display based on predefined operational states, including Normal, Warning, Critical, and Fire Alert. Each state is represented using a distinct visual layout to ensure clear communication of system status to the user. Particular emphasis is placed on intuitive visual cues, emergency prioritization, and readability under varying conditions. The chapter focuses on how sensor data is translated into meaningful visual feedback through structured display modes.

4.1 OLED Layout Modes

We implemented an OLED display (SSD1306, 128x64 pixels) that shows environmental data with different layouts based on the system state. Instead of a static display, the screen content adapts dynamically to prioritize the most critical information.

4.1.1 Normal State

In the **Normal** state (Temperature 15–30°C, Humidity 40–80%), the display shows a clean layout with full temperature and humidity values. It includes a comfort status text (IDEAL or GOOD) and a smiley icon to indicate safe conditions. This mode is designed for readability and reassurance.



Figure 4.1: OLED display in Normal state showing temperature, humidity, and comfort icon.

4.1.2 Warning State

The **Warning** state triggers when conditions drift outside the comfortable range (Temperature <15°C or >33°C, or Humidity <40% or >80%). The display features a blinking border (500ms interval) to catch the user's eye. It shows abbreviated sensor readings and a specific warning message (e.g., "HOT", "COLD", "DRY", "TOO HUMID") along with an exclamation icon.

4.1.3 Critical State

In the **Critical** state (Temperature $>40^{\circ}\text{C}$ or Humidity $<30\%$), the display switches to a high-urgency mode. The header is inverted and flashes to maximize contrast. The screen displays the sensor values along with a corrective action message (e.g., "TOO HOT! Cool down!" or "TOO DRY! Add moisture!"), guiding the user to take immediate steps.

4.1.4 Fire Alert State

The **Fire Alert** state is the highest priority. When a flame is detected, the screen ignores sensor values and flashes a full-screen emergency alert. It displays "FIRE DETECTED!" and "EVACUATE NOW" in large text. This mode is designed to be visible from a distance to ensure safety.



Figure 4.2: OLED display in Fire Alert state showing emergency evacuation message.

4.2 State Selection Flow

The OLED task uses a semaphore-based mechanism to decide which display function to execute. Instead of complex if-else chains checking global variables, the task simply checks which state semaphore is currently available. This ensures that the display logic is always synchronized with the system's core state machine.

The state evaluation logic enforces a priority hierarchy: **Fire Alert** takes precedence over **Critical**, which overrides **Warning**, with **Normal** being the default state. This guarantees that the most urgent information is always displayed.

The following pseudocode describes the selection flow:

```
FUNCTION updateDisplay():
    Acquire I2C Mutex

    // Check which state is active
    state = getSystemState()

    SWITCH state:
        CASE FIRE_ALERT:
            drawFireAlertDisplay(animationFrame)
            BREAK

        CASE CRITICAL:
            drawCriticalDisplay(temp, humi, animationFrame)
            BREAK

        CASE WARNING:
            drawWarningDisplay(temp, humi, animationState)
            BREAK

        CASE NORMAL:
            drawNormalDisplay(temp, humi)
            BREAK

    Release I2C Mutex

    // Adjust refresh rate based on urgency
    IF state >= WARNING:
        Delay 300ms // Fast refresh for alerts
    ELSE:
        Delay 1000ms // Save power in normal mode
END FUNCTION
```

4.3 Queue and Semaphore Data Flow

One of the main requirements was to remove all global variables and use RTOS primitives instead. We did this by putting all sensor readings into a `SensorData_t` structure that goes through the queue, and using semaphores for state information.

We removed global variables in three ways:

1. **Data in Queue:** All sensor values are stored in the `SensorData_t` struct in the queue, so we don't need separate global variables for temperature, humidity, etc.
2. **State with Semaphores:** We use binary semaphores to signal system states instead of global state variables
3. **Task Communication:** Tasks talk to each other through queues and semaphores, not shared memory, which makes it thread-safe

The OLED task demonstrates this pattern through its main loop structure:

```
TASK temp_humi_oled():
    Initialize OLED display (with I2C mutex protection)
    Initialize DHT11 sensor

    WHILE TRUE:
        Read temperature and humidity from DHT11
        Validate readings

        // Pack data into struct (no global variables)
        sensorData.temperature = temperature
        sensorData.humidity = humidity

        // Evaluate state using pure function
        newState = evaluateSystemState(temp, humidity, flame)

        // Update semaphores (thread-safe)
        updateSystemState(newState)

        // Send to queue for other tasks
        sendSensorData(sensorData)
```

```
// Render display based on state
Acquire I2C mutex
currentState = getSystemState()
SWITCH currentState:
    CASE FIRE_ALERT: drawFireAlertDisplay()
    CASE CRITICAL: drawCriticalDisplay()
    CASE WARNING: drawWarningDisplay()
    CASE NORMAL: drawNormalDisplay()
Release I2C mutex

// Adaptive delay based on urgency
IF state >= WARNING: Delay 300ms
ELSE: Delay 1000ms
END WHILE
END TASK
```

The state evaluation is implemented as a pure function with no side effects:

```
FUNCTION evaluateSystemState(temp, humidity, flame):
    IF flame: RETURN FIRE_ALERT
    IF temp > 40 OR humidity < 30: RETURN CRITICAL
    IF temp < 20 OR temp > 33 OR humidity < 40 OR humidity > 80:
        RETURN WARNING
    RETURN NORMAL
END FUNCTION
```

The state update function handles semaphore signaling with mutex protection, ensuring atomic state transitions:

```
FUNCTION updateSystemState(newState):
    Acquire state mutex
    IF newState != currentState:
        // Clear all semaphores atomically
        Take all state semaphores (timeout = 0)

        // Give only the active state semaphore
        SWITCH newState:
            CASE NORMAL: Give xSemaphoreNormal
            CASE WARNING: Give xSemaphoreWarning
```

```
CASE CRITICAL: Give xSemaphoreCritical  
CASE FIRE_ALERT: Give xSemaphoreFireAlert
```

```
    currentState = newState  
    Release state mutex  
END FUNCTION
```

This architecture completely eliminates global variable dependencies by using FreeRTOS queues for data passing and semaphores for state signaling, achieving deterministic thread-safe inter-task communication without shared memory vulnerabilities.

The OLED task works like this: it reads temperature and humidity from the DHT sensor, checks if the readings are valid, and puts them into a `SensorData_t` structure. This structure is sent to the queue so other tasks can read it, avoiding global variables.

To determine the system state, we call `evaluateSystemState()` which takes sensor readings and returns the state based on thresholds. This function doesn't use any global variables. Then we call `updateSystemState()` to update the semaphores.

The `updateSystemState()` function works atomically: it first takes a mutex, then clears all state semaphores by taking them with zero timeout, and finally gives only the semaphore for the new state. This way, only one state semaphore is available at a time, so other tasks can check which state is active.

For displaying, the task takes the I2C mutex before writing to the OLED, calls `getSystemState()` to check which state semaphore is available, and then calls the right display function. We use different delays: 300 ms during warnings/critical states for faster updates, and 1000 ms during normal operation to save CPU.

Chapter 5

Web Server Interface and Cloud Integration

This chapter details the implementation of the user interface and cloud connectivity features. We designed a dual-mode network architecture where the ESP32 operates simultaneously as an Access Point (AP) for local control and a Station (STA) for cloud data publishing. This hybrid approach ensures that the device remains controllable locally even when internet connectivity is lost.

5.1 Embedded Web Server Design

We developed a responsive web interface hosted directly on the ESP32's flash memory using the LittleFS file system. The web server is built on the `ESPAsyncWebServer` library, which provides asynchronous request handling to maintain system responsiveness even under heavy network load.

5.1.1 Access Point Configuration

To ensure the device is always accessible, we configured the ESP32 to broadcast its own WiFi network. This allows users to connect directly to the device without needing an external router.

- **SSID:** `YoloUNO_AP`
- **IP Address:** `192.168.4.1` (Static)
- **Security:** WPA2-PSK (Password protected)
- **Mode:** `WIFI_AP_STA` (Simultaneous AP + Station)

The network initialization logic in `task_wifi.cpp` ensures that the AP interface remains active even while the device attempts to connect to an external WiFi network. This "AP-first" strategy prevents the device from becoming unreachable if the home WiFi credentials change or the router goes offline.

5.1.2 User Interface Implementation

The user interface is implemented using standard HTML5, CSS3, and JavaScript, served as static files from the device's filesystem. This separation of concerns allows us to update the UI look-and-feel without recompiling the firmware.

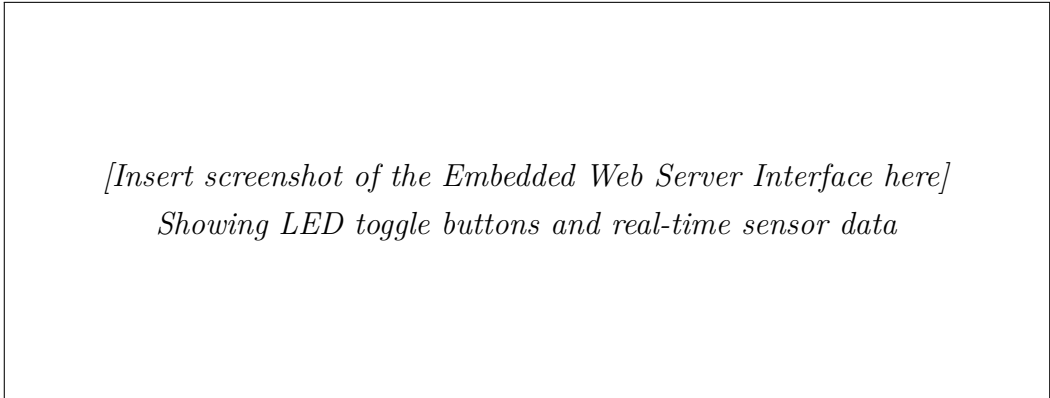
The interface features a dashboard layout with two primary control sections:

1. Device Control Panel:

- **LED Control:** A toggle switch to control the onboard LED (GPIO 48).
- **Fan/Pump Control:** A secondary toggle for the high-power actuator (GPIO 2 or Pump).

2. Real-time Status:

- **Sensor Readings:** Live updates of temperature, humidity, and light levels.
- **Connection Status:** Indicators for WiFi signal strength and Cloud connectivity.



*[Insert screenshot of the Embedded Web Server Interface here]
Showing LED toggle buttons and real-time sensor data*

Figure 5.1: Embedded Web Server interface showing control buttons and sensor status.

5.1.3 Web Communication Protocols

HTTP vs. WebSocket

Traditional web servers use the HTTP protocol, which follows a request-response model. The client requests data, the server responds, and the connection is closed. This is inefficient for real-time control because the client must constantly poll the server to get updates.

For our control interface, we chose the **WebSocket** protocol. WebSocket provides a full-duplex communication channel over a single TCP connection. Once established, the server can push data to the client immediately without waiting for a request. This significantly reduces latency and network overhead, making it ideal for controlling actuators (like LEDs) and displaying live sensor data.

5.1.4 WebSocket Communication

To achieve real-time performance without page reloads, we utilized the WebSocket protocol. Unlike traditional HTTP polling, WebSocket establishes a persistent, full-duplex connection between the browser and the ESP32.

```
1 void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
2             AwsEventType type, void *arg, uint8_t *data, size_t len) {
3     if (type == WS_EVT_DATA) {
4         AwsFrameInfo *info = (AwsFrameInfo *)arg;
5         if (info->opcode == WS_TEXT) {
6             String message = String((char *)data).substring(0, len);
7
8             // Parse JSON command
9             // Example: {"action":"toggleLed", "value":true}
10            handleWebSocketMessage(message);
11        }
12    }
13 }
```

Listing 5.1: WebSocket event handler for real-time control

When a user clicks a button on the web interface, a JSON message is sent over the WebSocket. The ESP32 parses this message and triggers the corresponding FreeRTOS task (e.g., giving a semaphore to the LED task). Conversely, when sensor values change, the ESP32 broadcasts the new data to all connected clients immediately.

[Insert diagram of WebSocket Communication Flow here]
Client -> JSON Command -> ESP32 -> Task Notification

Figure 5.2: WebSocket communication flow for real-time device control.

5.2 Cloud Integration and Data Publishing

While the local web server provides immediate control, long-term data storage and remote access are handled by the CoreIOT cloud platform.

5.2.1 Station Mode Connectivity

The device connects to the local WiFi network in Station (STA) mode. We implemented a robust reconnection state machine that:

1. Attempts to connect to the configured WiFi SSID.
2. If connection fails, it waits for a backoff period before retrying.
3. Periodically checks connection health and triggers reconnection if the link is dropped.
4. Synchronizes system time via NTP (Network Time Protocol) upon successful connection.

Crucially, the reconnection logic runs in a low-priority background task to avoid blocking critical sensor processing.

5.2.2 Data Publishing to CoreIOT

We utilize the MQTT protocol to publish sensor telemetry to the CoreIOT server (`app.coreiot.io`). The publishing logic is encapsulated in the `CORE_IOT_task`, which aggregates data from the shared sensor queue and transmits it securely.

Authentication Configuration: The connection is authenticated using a unique Device Token. This token acts as the MQTT username, ensuring that data is routed to the correct device dashboard in the cloud.

```
1 // CoreIOT Configuration
2 #define CORE_IOT_SERVER "app.coreiot.io"
3 #define CORE_IOT_PORT 1883
4 #define DEVICE_TOKEN "YOUR_DEVICE_TOKEN_HERE" // Replaced with actual
   token
5
6 void publishTelemetry() {
7     if (tb.connected()) {
8         // Aggregate latest readings
9         SensorData_t data;
10        getSensorData(&data);
11
12        // Publish to cloud
13        tb.sendTelemetryData("temperature", data.temperature);
14        tb.sendTelemetryData("humidity", data.humidity);
15        tb.sendTelemetryData("light", data.light_level);
16
17        Serial.println("[Cloud] Data published");
18    }
19 }
```

Listing 5.2: CoreIOT authentication and data publishing

5.2.3 Integration Improvements

Building upon the initial setup described in Chapter 1, we enhanced the cloud integration with several reliability features:

- **Data Buffering:** Sensor readings are buffered in the FreeRTOS queue, ensuring that the latest valid reading is always available for publishing, even if the sensor task is temporarily busy.
- **Connection Resilience:** The system automatically handles MQTT broker disconnections without losing state. It attempts to reconnect to the cloud independently of the WiFi connection status.
- **Traffic Optimization:** Telemetry is published at a fixed interval (10 seconds) rather than on every sensor change, reducing bandwidth usage and complying with CoreIOT rate limits.

This dual-interface approach—local Web Server for low-latency control and CoreIOT Cloud for historical analysis—provides a comprehensive user experience that combines the best aspects of edge and cloud computing.

5.3 Data Logging and CSV Export

To facilitate offline analysis and historical data tracking, we implemented a local data logging feature. The system periodically records sensor metrics and system states to a CSV (Comma-Separated Values) file stored in the ESP32's flash memory using the LittleFS file system.

5.3.1 CSV File Structure

The log file is named `data.csv` and contains the following columns:

- **datetime:** Human-readable timestamp (YYYY-MM-DD HH:MM:SS)
- **timestamp:** Unix timestamp for machine processing
- **temperature, humidity:** Environmental readings
- **light_level, moisture_level:** Analog sensor values

- **flame_detected, water_pump:** Boolean status flags (0/1)
- **system_state:** Textual representation of the system's operating mode

5.3.2 System State Classification

The **system_state** column provides a high-level summary of the device's condition at the time of logging. This value is derived from the **SystemState_t** enum and is exported as a string to ensure readability. Table ?? lists the possible values and their corresponding conditions.

Table 5.1: System State values in the exported CSV file.

CSV Value	Enum Value	Condition
Normal	STATE_NORMAL (0)	Temperature 15–33°C, Humidity 50–70%
Warning	STATE_WARNING (1)	Temperature 33–40°C or Humidity out of ideal range
Critical	STATE_CRITICAL (2)	Temperature > 40°C or Humidity < 30%
Fire	STATE_FIRE_ALERT (3)	Flame sensor detected fire source

Users can download this file directly from the web interface for analysis in spreadsheet software like Microsoft Excel or Google Sheets.

Chapter 6

Chapter 6

This chapter covers...

6.1 Section 1

Content for section 1...

6.2 Section 2

Content for section 2...

6.3 Summary

Summary of chapter 6...

Chapter 7

Task 6 - Data Publishing to CoreIOT Cloud Server

This chapter documents the process of setting up the Core IOT platform for our IoT project. We cover account creation, device registration, and dashboard configuration.

7.1 Create Core IOT Account

Core IOT is a cloud-based IoT platform built on the ThingsBoard framework. We selected this platform for our project due to its open-source nature and comprehensive feature set for device management and data visualization.

7.1.1 Registration Process

We accessed the Core IOT platform at <https://coreiot.io> and clicked the "Start Now" button to begin registration (Figure ??).



assets/Coreiot.png

Figure 7.1: Core IOT homepage with the "Start Now" button for registration.

The platform offers multiple login options through OAuth providers. We chose to authenticate using a Google account for convenience (Figure ??).



assets/Login.png

Figure 7.2: Login interface with OAuth provider options.

After successful authentication, we were assigned the Tenant Administrator role, which provides full access to device management and dashboard creation features (Figure ??).



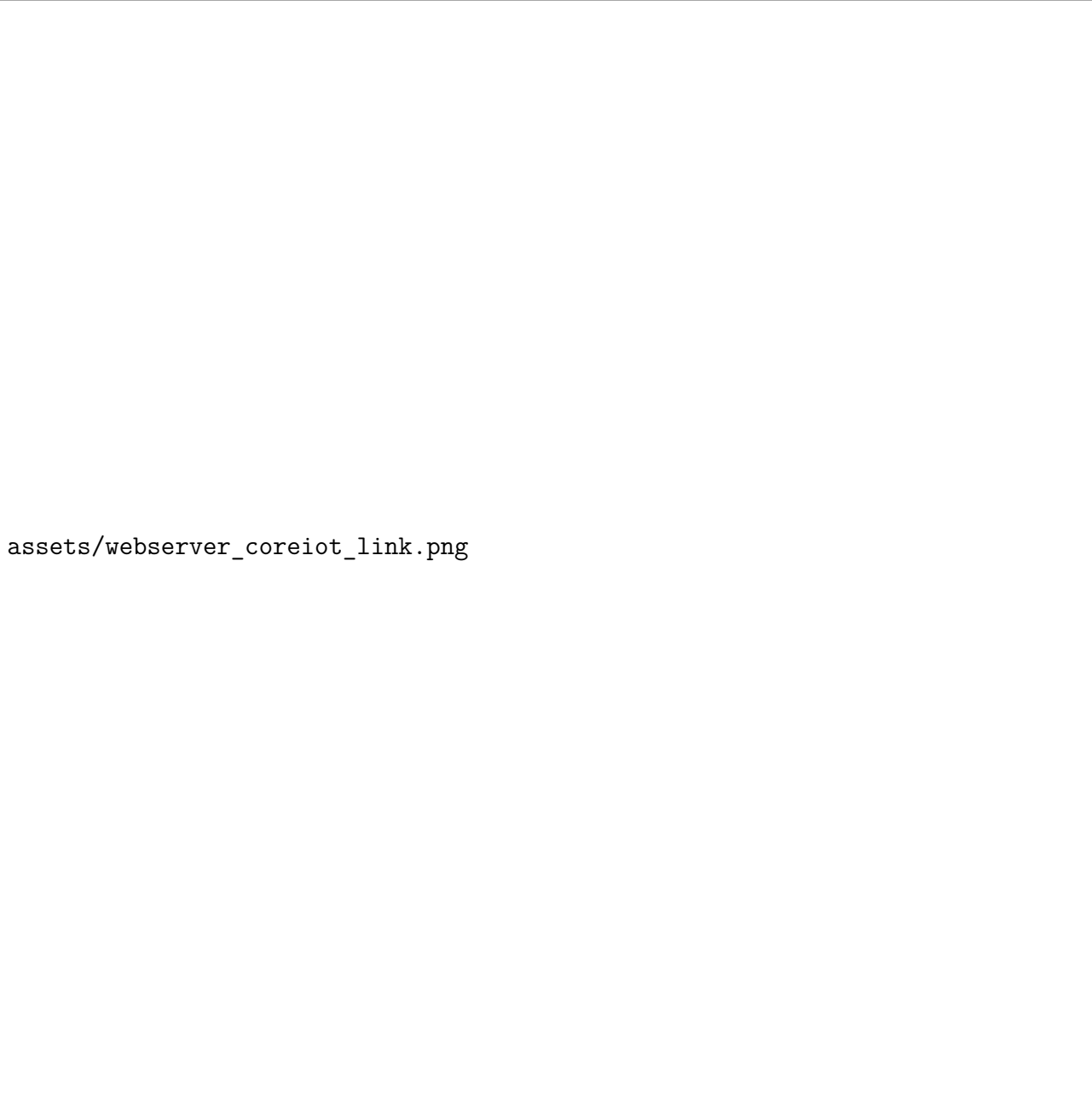
assets/platform.png

Figure 7.3: Dashboard view after login showing Tenant Administrator role.

7.2 Accessing CoreIOT via Web Server

To provide a seamless user experience, we integrated a direct link to the CoreIOT dashboard within our local web server interface. On the home page of the device's web interface (detailed in Chapter 5), we added a dedicated "card" that redirects users to the cloud dashboard.

This integration allows users to easily transition from local control (Access Point mode) to remote monitoring (Cloud Dashboard) without needing to manually type the URL.



assets/webserver_coreiot_link.png

Figure 7.4: Web server home page showing the CoreIOT access card.

7.2.1 Device Provisioning

With the account ready, we proceeded to register our ESP32-based device on the platform. In the "Devices" section, we created a new device and obtained an access token. This token is required for the device to authenticate when sending telemetry data via MQTT.

In our firmware, we defined global variables to store the connection credentials. These values are loaded from non-volatile storage during boot, allowing users to configure them through our web interface without modifying the source code:

```
1 // WiFi & CoreIOT Configuration
2 extern String WIFI_SSID;
3 extern String WIFI_PASS;
4 extern String CORE_IOT_TOKEN;
5 extern String CORE_IOT_SERVER;
6 extern int CORE_IOT_PORT;
```

Listing 7.1: Global configuration variables for Core IOT connection (global.h)

The connection to the Core IOT server is established using the ThingsBoard Arduino SDK. We initialize the MQTT client and ThingsBoard object as follows:

```
1 constexpr uint32_t MAX_MESSAGE_SIZE = 1024U;
2
3 WiFiClient wifiClient;
4 Arduino_MQTT_Client mqttClient(wifiClient);
5 ThingsBoard tb(mqttClient, MAX_MESSAGE_SIZE);
```

Listing 7.2: MQTT client initialization (task_core_iot.cpp)

The reconnection logic handles authentication and subscription setup. When the device connects successfully, it sends its MAC address as an attribute and subscribes to RPC callbacks:

```
1 void CORE_IOT_reconnect() {
2     if (CORE_IOT_TOKEN.isEmpty() || CORE_IOT_SERVER.isEmpty())
3         return;
4
5     if (!tb.connected()) {
6         if (!tb.connect(CORE_IOT_SERVER.c_str(),
7                         CORE_IOT_TOKEN.c_str(),
8                         CORE_IOT_PORT))
9             return;
10
11         Serial.println("[CoreIOT] Connected");
12         tb.sendAttributeData("macAddress", WiFi.macAddress().c_str());
13         tb.RPC_Subscribe(callbacks.cbegin(), callbacks.cend());
14         tb.sendAttributeData("localIp", WiFi.localIP().toString().c_str())
15         ;
16     } else {
17         tb.loop();
18     }
```

Listing 7.3: Core IOT reconnection and subscription logic

To verify the connection and data transmission capabilities, we utilized a Python script to simulate the IoT device behavior before deploying it to the actual hardware. It is critical to

note that we specifically selected the `paho-mqtt` library version 1.6.1 for this implementation, as newer versions (e.g., 2.0) introduce breaking changes in the authentication handling that are incompatible with our current configuration.

7.3 Telemetry Data Transmission

After configuring the device credentials, we developed a firmware routine to format sensor data into a JSON structure compliant with the Core IOT platform's requirements. We defined a struct to hold all sensor values in a thread-safe manner:

```
1 typedef struct {
2     float temperature;
3     float humidity;
4     float light_level;
5     float moisture_level;
6     bool flame_detected;
7     bool fan_enabled;
8     bool neoled_enabled;
9     bool water_pump_enabled;
10 } SensorData_t;
```

Listing 7.4: Sensor data structure for telemetry (`global.h`)

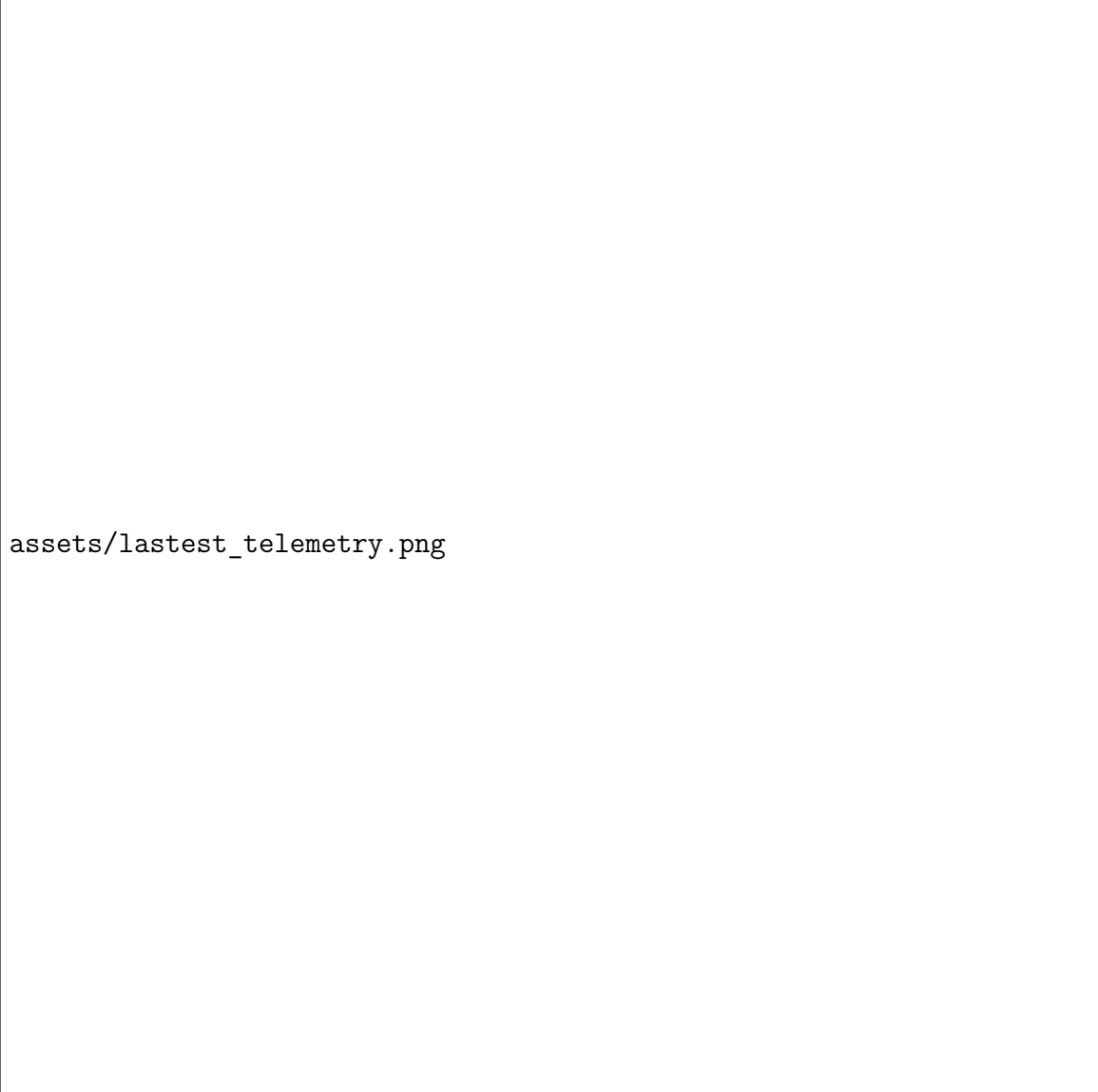
The main task runs in a FreeRTOS task, waiting for WiFi connectivity before attempting to connect to Core IOT. It then enters a loop that publishes telemetry data every 10 seconds:

```
1 void CORE_IOT_task(void *pvParameters) {
2     // Wait for WiFi connection
3     while (1) {
4         if (xSemaphoreTake(xBinarySemaphoreInternet, portMAX_DELAY)) {
5             Serial.println("[CoreIOT] WiFi ready");
6             break;
7         }
8         vTaskDelay(500 / portTICK_PERIOD_MS);
9     }
10
11     unsigned long lastPublish = 0;
12     const unsigned long PUBLISH_INTERVAL = 10000;
13     SensorData_t sensorData = {0};
14
15     while (1) {
16         CORE_IOT_reconnect();
17
18         if (millis() - lastPublish >= PUBLISH_INTERVAL) {
19             lastPublish = millis();
```

```
20         getSensorData(&sensorData);  // Thread-safe read
21
22         if (tb.connected()) {
23             tb.sendTelemetryData("temperature", sensorData.temperature
24         );
25             tb.sendTelemetryData("humidity", sensorData.humidity);
26             tb.sendTelemetryData("light", sensorData.light_level);
27             tb.sendTelemetryData("moisture", sensorData.moisture_level
28         );
29         }
30     }
31     vTaskDelay(500 / portTICK_PERIOD_MS);
32 }
```

Listing 7.5: Telemetry publishing routine in the main task loop

We verified the data ingestion by navigating to the "Latest Telemetry" tab within the device details page on the Core IOT portal. This interface allows us to confirm that the JSON payload is being parsed correctly and that values for temperature and humidity are updating dynamically as sent by the device.



assets/lastest_telemetry.png

Figure 7.5: The "Latest Telemetry" tab displaying real-time data received from the device.

7.4 Dashboard Implementation

One of the primary advantages of the Core IOT platform is its capability to visualize data through customizable dashboards. We created a comprehensive monitoring dashboard named *Control and Monitoring center* to serve as the central interface for real-time system oversight and actuator control. The dashboard employs a multi-panel layout that organizes information hierarchically, prioritizing critical control functions and environmental data visualization.

7.4.1 Dashboard Layout Architecture

We designed the dashboard with a responsive grid layout divided into three primary horizontal sections. The top section contains system status indicators and actuator controls, providing immediate access to the most frequently used functions. The middle section displays real-time sensor data through time-series charts and gauge widgets, enabling trend analysis and anomaly detection. The bottom section presents an alarm management table and geographical visualization, supporting incident tracking and multi-device deployment scenarios.

This hierarchical organization follows established human-computer interaction principles, placing high-priority interactive elements within the primary visual field while maintaining comprehensive data visibility through strategic use of white space and visual grouping.

7.4.2 Status Indicators and Device Metrics

The dashboard header displays two key performance indicators using card widgets. The first card shows the active device count, labeled **Devices**, which indicates the number of ESP32 devices currently connected to the platform. In our deployment, this displays a value of 1, confirming that our primary environmental monitoring device maintains an active MQTT connection.

Adjacent to the device count, we positioned a **Total** card that aggregates a custom metric across all connected devices. This counter widget can be configured to track various cumulative statistics such as total alarm count, cumulative uptime, or aggregated sensor readings. The card widgets utilize ThingsBoard's entity aggregation functions, which query the time-series database to compute values based on configurable time windows.

To the right of the status cards, we integrated a WiFi quality indicator using an icon widget. This widget dynamically changes color based on the RSSI (Received Signal Strength Indicator) value transmitted in the device telemetry. Strong signal strength displays a green icon, while degraded connectivity transitions to yellow or red, providing immediate visual feedback on network health without requiring detailed inspection of raw telemetry values.

7.4.3 Remote Control Panel

We implemented a dedicated control panel section containing three RPC toggle switches for actuator management: Water Pump, Fan, and NeoLed (RGB LED strip). Each switch widget is configured with bidirectional RPC methods to both read the current state and issue control commands.

The switch configuration employs getter and setter RPC methods. When the dashboard loads, each switch automatically invokes its corresponding getter method (`getFanStatus`, `getWaterPumpStatus`, `getLedStatus`) to synchronize the UI state with the actual device state.

This prevents desynchronization scenarios where the dashboard might display **OFF** while the device is actively driving the actuator.

When a user toggles a switch, the dashboard transmits an RPC request containing the method name and the desired boolean state. For example, clicking the Fan switch from OFF to ON triggers an RPC call to `setFanStatus` with parameter `true`. The device firmware processes this command, actuates the corresponding GPIO pin, and returns an `RPC_Response` to acknowledge successful execution.

We observed that timeout errors occasionally occur when the device is processing intensive tasks or when network latency exceeds the default RPC timeout threshold (5 seconds). In such cases, the switch widget displays a *Request timeout* modal, prompting the user to retry the command. To mitigate this issue, we optimized the RPC callback functions to execute GPIO operations quickly and defer any blocking operations to separate FreeRTOS tasks.

7.4.4 Environmental Data Visualization

The central focus of the dashboard is the environmental monitoring section, which utilizes time-series charts to visualize temperature and humidity trends. We configured a line chart widget with dual Y-axes, where temperature is plotted in degrees Celsius on the left axis and relative humidity percentage on the right axis. This dual-axis configuration enables direct visual correlation between the two variables, which is essential for identifying environmental patterns such as evaporative cooling effects.

The chart widget subscribes to the `temperature` and `humidity` telemetry keys and maintains a rolling time window of the last hour by default. Users can adjust this window through the chart's time range selector, enabling both real-time monitoring and historical analysis. We customized the chart appearance with distinct colors: red for temperature and blue for humidity, following conventional meteorological visualization standards.

Below the time-series chart, we positioned a second chart dedicated to soil moisture monitoring. This widget displays the `moisture` telemetry key, providing agricultural and plant care applications with a clear view of substrate hydration levels over time. The moisture chart uses a green color scheme and includes threshold lines at 30% (dry) and 70% (saturated) to guide irrigation decisions.

Adjacent to the charts, we implemented a large numeric display widget showing the current light level in lux. This widget is configured with the `light` telemetry key and formatted to display values with scientific notation (e.g., 65000 m², though the unit should technically be lux). The large font size makes this value readable at a glance, which is particularly useful for automated greenhouse or indoor farming applications where lighting optimization is critical.

7.4.5 Alarm Management System

We integrated a comprehensive alarm table widget in the lower section of the dashboard to track environmental anomalies and system events. The alarm system operates through rule chains configured in the ThingsBoard Rule Engine, which continuously evaluate incoming telemetry against predefined threshold conditions.

The alarm table displays several critical columns:

- **Created time:** Timestamp when the alarm was triggered, enabling chronological incident analysis
- **Originator:** The entity that generated the alarm (typically *Assignment* for our configured rule chain)
- **Type:** Classification of the alarm event (e.g., *Environment Warning*)
- **Severity:** Criticality level (Warning, Critical, or Fire Alert)
- **Status:** Current state of the alarm (Active, Acknowledged, Cleared, Unacknowledged)
- **Assignee:** User or group responsible for incident response

In our current deployment, the alarm table shows multiple *Environment Warning* entries triggered on December 3, 2025. These alarms are generated when temperature exceeds 33°C, drops below 15°C, or when humidity falls outside the 40–70% comfort range. The severity levels are color-coded: green for acknowledged alarms, yellow for warnings, and red for critical events.

We configured the alarm rules using ThingsBoard's Rule Chain editor, which provides a visual programming interface for defining conditional logic. For example, the temperature threshold rule consists of a filter node that evaluates `temperature > 33`, connected to an action node that creates an alarm entity with severity *Warning* and type *Environment Warning*. The rule chain also sends WebSocket notifications to the dashboard, triggering real-time alert pop-ups.



assets/rule_chain.png

Figure 7.6: Rule Chain configuration for alarm processing and notification.

The alarm table supports interactive management features. Operators can click individual alarms to view detailed information, acknowledge events to indicate they are being addressed, or assign alarms to specific team members for resolution. This functionality is particularly valuable in multi-user deployments where incident response requires coordination among maintenance personnel.

7.4.6 Widget Configuration

Throughout the dashboard design process, we identified several critical configuration requirements:

Data Key Consistency: The telemetry keys defined in the firmware (`temperature`, `humidity`, `light`, etc.) must exactly match the data keys configured in each widget. Case sensitivity and spelling errors will result in widgets displaying *No data* even when the device is actively transmitting.

RPC Method Naming: Switch widgets require precise RPC method names that correspond to the registered callbacks in the firmware. We standardized on a naming convention using camelCase with *set* and *get* prefixes (e.g., `setFanStatus`, `getFanStatus`).

Widget Refresh Rates: We configured chart widgets to poll for new data every 5 seconds, balancing real-time responsiveness with server load. Faster refresh rates increase database query frequency, which can impact platform performance in deployments with hundreds of devices.

Color Coding and Visual Hierarchy: We applied consistent color schemes throughout the dashboard: green for normal conditions, yellow for warnings, and red for critical alerts. This visual language enables rapid situation assessment without requiring detailed reading of all numeric values.

7.5 Summary

In this chapter, we successfully set up the Core IOT platform for our project. We created a Tenant Administrator account, registered our ESP32 device, and established MQTT communication for telemetry data transmission. The firmware implementation includes connection handling with automatic reconnection, periodic telemetry publishing using FreeRTOS tasks, and bidirectional RPC communication for remote device control. The dashboard was configured with multiple widget types including Digital Gauges for real-time sensor readings, Charts for historical trends, RPC Switches for remote device control, and an Open Street Map for geographical visualization. These components form the foundation for the monitoring and control system that will be expanded in subsequent chapters.

Chapter 8

Chapter 8

This chapter covers...

8.1 Section 1

Content for section 1...

8.2 Section 2

Content for section 2...

8.3 Summary

Summary of chapter 8...

Chapter 9

Chapter 9

This chapter covers...

9.1 Section 1

Content for section 1...

9.2 Section 2

Content for section 2...

9.3 Summary

Summary of chapter 9...

Conclusion

Summary

This section summarizes the key findings and contributions of this project...

Future Work

Future work and potential improvements...

Final Remarks

Final remarks and closing thoughts...

Project Resources

The complete source code and project documentation can be accessed via the following links:

- **Source Code (GitHub):** <https://github.com/your-username/your-repo>
- **Project Data Report (Google Drive):** <https://drive.google.com/drive/folders/your-folder-id>